

LLM Evals: The Pragmatist's Guide

> Moving from 'Vibes-Based' Development to Evidence-Based Engineering

Based on "LLM Evals: Everything You Need to Know"
by Hamel Husain & Shreya Shankar

```
101 {
102     "timestamp": "2024-05-15T10:30:00Z",
103     "level": "INFO",
104     "message": "Model output received",
105     "data": {
106         "completion": "...",
107         "token_usage": {
108             "prompt_tokens": 120,
109             "completion_tokens": 256,
110             "total_tokens": 376
111         }
112     },
113     "el output received",
114     "token_usage": {
115         "prompt_tokens": 120
116         "total_tokens": 376
117     }
118 }
119
120 File "/src/evals/metrics.py", line 42, in compute_score
121     result = evaluate_function(input_data, model_output)
122 File "/src/evals/utils.py", line 101, in evaluate_function
123     raise ValueError("Invalid output format")
124 ValueError: Invalid output format_
```


The Evaluation Playbook

1 | `01_The_Mindset.py` </>
Why error analysis eats infrastructure for lunch.

2 | `02_The_Method.py` </>
From "Open Coding" to Binary Grading.

3 | `03_The_Scale.py` </>
Structured Synthetic Data & LLM-as-a-Judge.

4 | `04_The_Stack.py` </>
Building Custom Tools & Debugging RAG.

5 | `05_Production.py` </>
Guardrails vs. Evaluators.

The Model is Not the Product

60–80% of development time in successful projects is spent on debugging and manual review.

Error Analysis (Looking at Data)

70%

Infrastructure & Modeling

Warning

The Trap: Buying expensive enterprise tools before understanding your failure modes.

The Fix: Spend 30 minutes manually reviewing 20–50 traces per iteration.

Start Small, Stay Human

The Wrong Way



Committee of Annotators / Outsourcing

Creates noise. Lack of shared context.
Slow alignment.

The Right Way

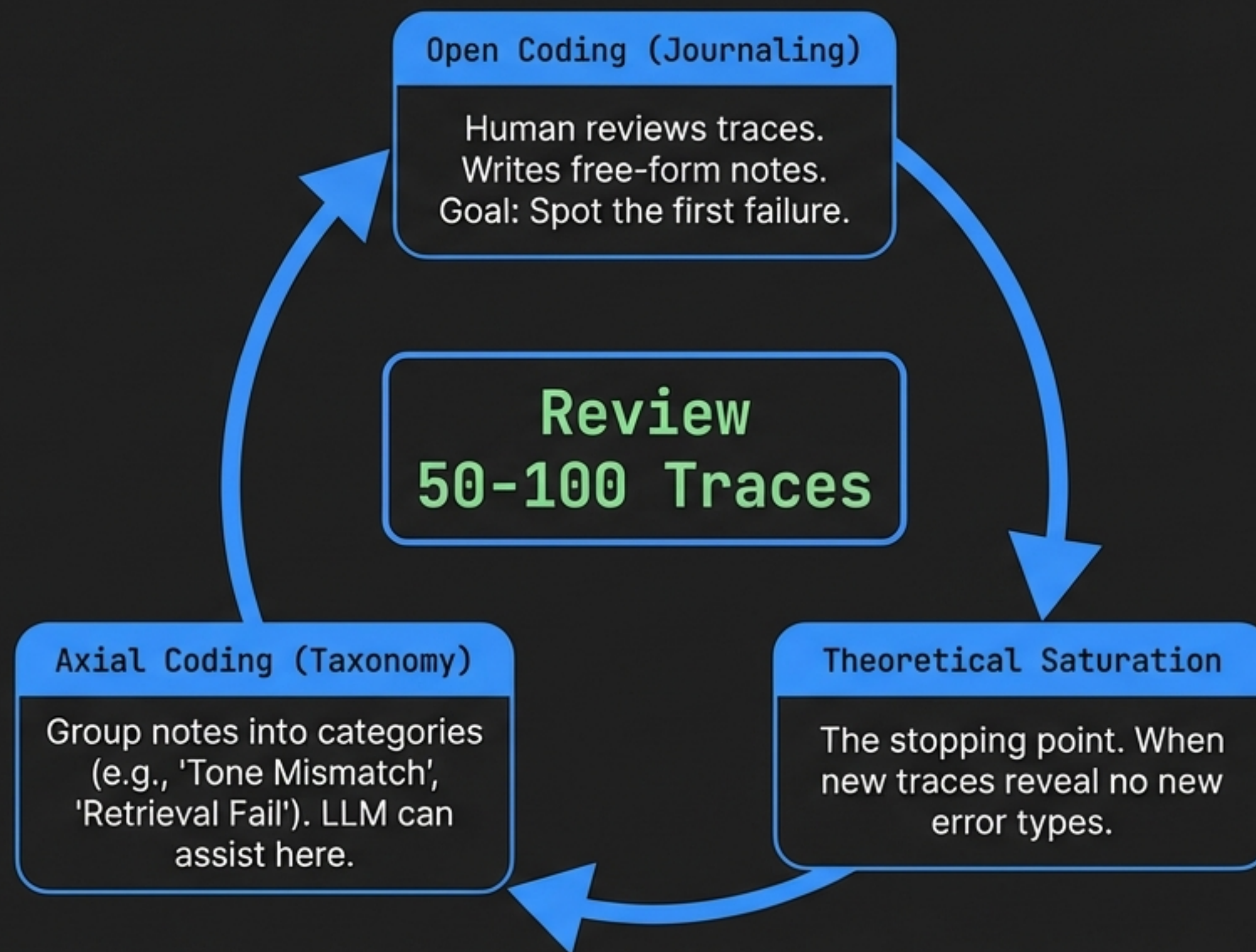


One Domain Expert

Single source of truth. Defines "taste"
and quality standards. Fast decisions.

Only scale to multiple annotators when the domain demands it (e.g., specialized medical RAG).

The Analysis Loop



Make Hard Decisions: Binary vs. Likert

Avoid Likert Scales

Rate Factual Accuracy: ★★★★★

Subjective. Is this a 3 or a 4?
Annotators hide in the middle.

Use Binary Assertions

- ☐ Did it include 4 out of 5 facts? → NO
- ☒ Did it mention the price? → YES

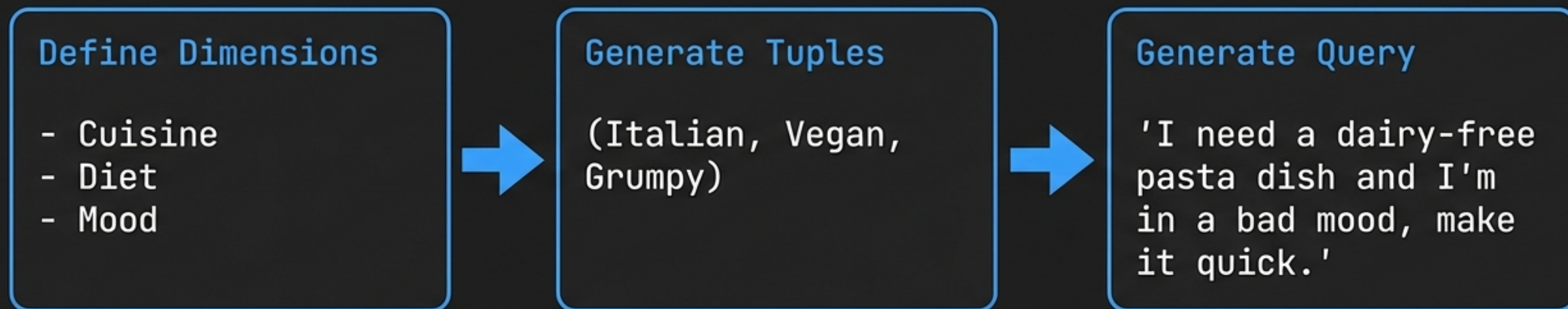
Forces clarity. If it's ambiguous,
the criteria is wrong.

Break complex subjectivity into multiple binary assertions.

16
17
18

Structured Synthetic Data

Synthesize the structure, then generate the text.



Avoid asking LLMs for **“random queries”**. This results in **generic trash**.
Avoid asking LLMs for “random queries”. This results in generic trash.

16
17
18

LLM-as-a-Judge

The Rule

Golden Rule: Never use off-the-shelf metrics like '**Helpfulness**' or '**Coherence**'. Build **custom judges** for **specific failure modes**. ⚠️

The Validation

Judge vs. Human

		Human Label Positive	Human Label Negative
Judge vs.	Judge Predicted Positive	True Positive	False Positive
	Judge Predicted Negative	False Negative	True Negative

Must verify Judge against Human labels. Measure **TPR** (True Positive Rate) and **TNR** (True Negative Rate) on a held-out set.

Side Note: Use the most capable model (GPT-4 class) for the judge.

Build Your Own Tools

High Velocity - Labeling

Trace ID: 8921 | Progress: 45/100

```
<div class='email-body'>
  <p>Your subscription has been
  updated.</p>
  <code>Order ID: #5592</code>
</div>
```

Context / Reasoning

User Input: "Why was
my plan changed?"

Tool Call:

lookup_subscription(
'user_id_123')

Tool Call:

target now. ...

Inter

Hotkeys: [J] Pass | [K] Fail | [L] Edit

- **Native Rendering** (Code as code, Email as email).
- **Keyboard Shortcuts** for flow.
- **Context aware** (User input + Tool calls).

Insight: Custom tools allow teams to iterate 10x faster.

Evaluating RAG

It's just Search + Generation.

RAG System

Retrieval (Search Engine)

- Metrics: **Precision, Recall, MRR.**
- Goal: **Finding the right document.**

Generation (LLM)

- Metrics: **Faithfulness, Hallucination, Relevance.**
- Goal: **Using the document correctly.**

Chunking Strategy

Fixed Output (Extract a number) -> **Large Chunks.**

Expansive Output (Summarize all) -> **Small Chunks.**

Debugging Agents

Transition Failure Matrix

	Success	Fail
Start		
Tool Selection		Hotspot
Parameter Extraction		
Execution		

Technique: N-1 Testing.
Feed the agent the first N-1 turns of a real failure, then test only the NEXT step.

Advice: Annotate the FIRST failure in a trace. Ignore cascading errors.

Production: Guardrails vs. Evaluators

Guardrails (The Bouncer)

- **Timing:** Synchronous (Pre-output).
- **Role:** Blocks harm, PII, profanity.
- **Tech:** Regex, Classifiers.
- **Speed:** Fast & Deterministic.

Evaluators (The Critic)

- **Timing:** Asynchronous (Post-output).
- **Role:** Measures quality, nuance.
- **Tech:** LLM-as-a-Judge.
- **Speed:** Slow & Expensive.

Critical: Never use a slow LLM-as-a-judge as a synchronous guardrail.

The Evals Manifesto

Error Analysis > Generic Metrics

Binary Labels > 1-5 Likert Scales

Internal Experts > Outsourced Labeling

Custom Tools > Off-the-shelf Platforms

Specific Failure Modes > 'Vibes' > _

The Model is NOT the Product. █

79

Dig Deeper

This guide is based on "LLM Evals: Everything You Need to Know" by Hamel Husain & Shreya Shankar.



Read the full blog post or join the course for code snippets.

80
81
82
83
84

>_ Presentation generated based on source material.